

Skip List – Create Node And Create Skip List

–Program Analysis

Program:

```
#define MAX_LEVEL 3

// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};

// Skip List structure
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};

// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    // elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
    + 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

```
// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

...
int main() {
    // Seed random number generator for random levels
    ...

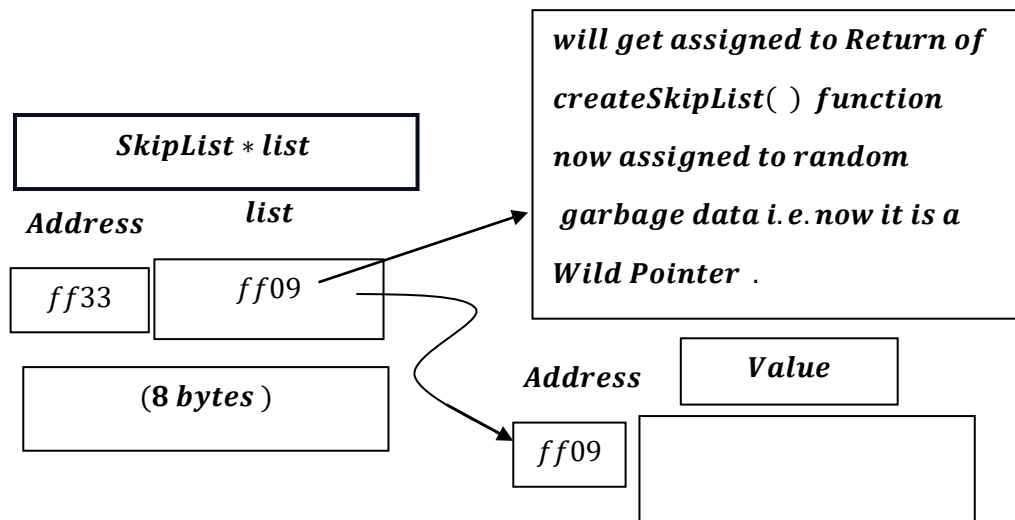
    struct SkipList *list = createSkipList();
...
}
```

Program Analysis

```
struct SkipList *list = createSkipList();
```

*Now, we have *list which is a pointer variable of struct SkipList ,
hence *list allocate 8 bytes ,which can access member variable of
struct SkipList → int level and header pointer.*

```
// Skip List structure
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```



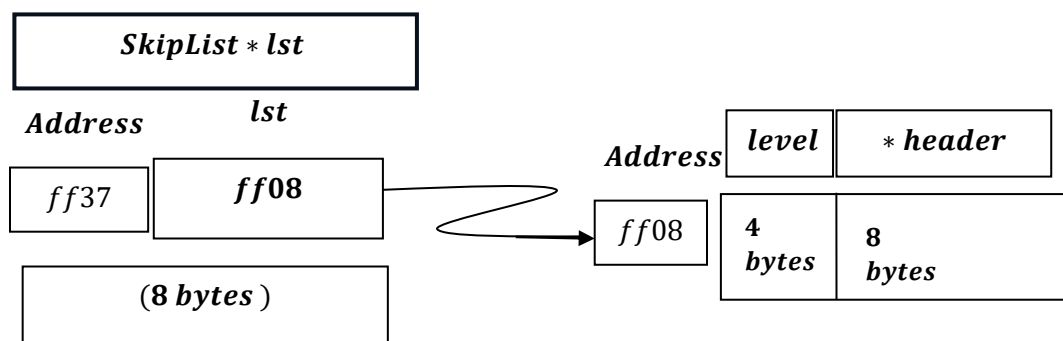
```
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Now,

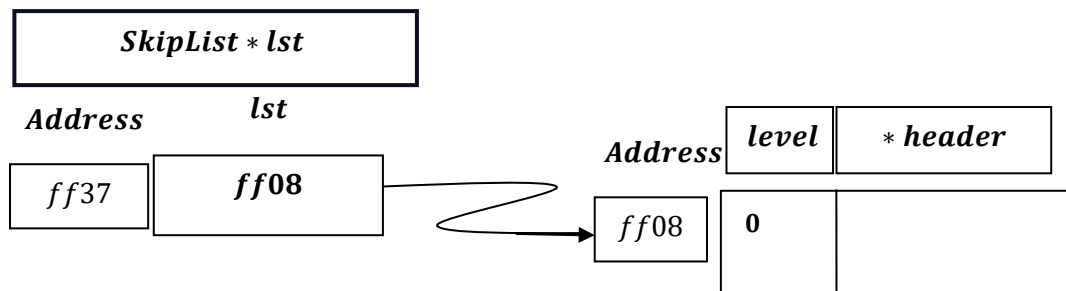
```
struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct SkipList));
```

We have `*lst` which is a pointer variable of struct `SkipList`, hence `*lst` allocate 8 bytes ,which can access member variable of struct `SkipList` → int level and header pointer.



```
lst->level = 0;
```

list → *level* is assigned to 0 .



```
lst->header = createNode(-1, MAX_LEVEL);
```

header will assigned to 8 bytes pointer variable which is an address return by *createNode(-1, MAX_LEVEL)* passing values - 1, and *MAX_LEVEL* = 3 .

```
// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
```

```

    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

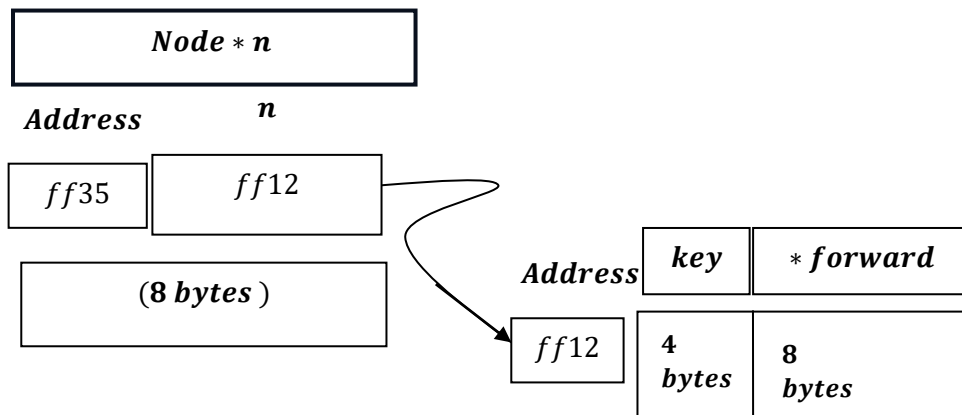
```

Now,

```

struct Node *n = (struct Node *)malloc(sizeof(struct Node));

```



```

struct Node *createNode(int key, int level) {

```

```

    n->key = key;

```

*This 'key' is
local variable.*

*This 'key' is
member variable
of struct Node.*

$n \rightarrow key = -1.$

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

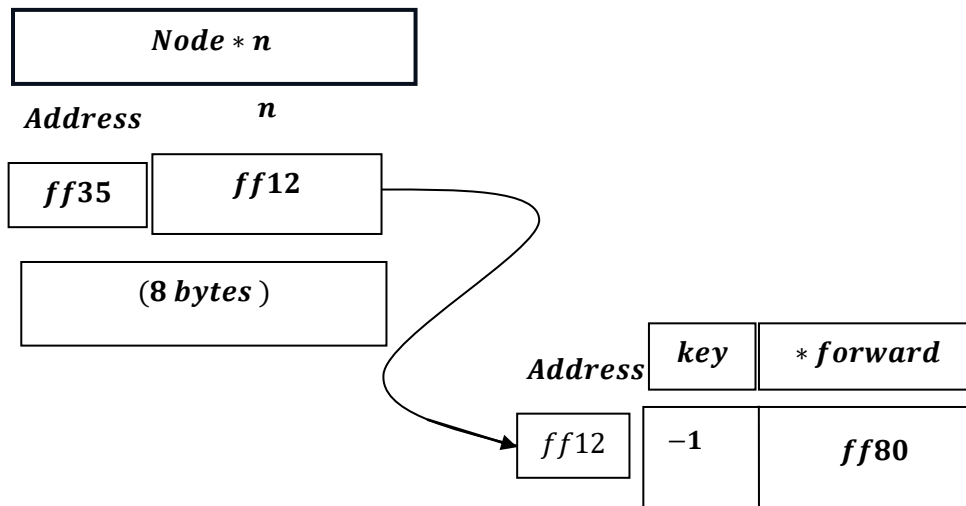
*It will allocate, $(struct Node * = 8 \text{ bytes}) \times (level + 1)$*

$= 8 \text{ bytes} \times (3 + 1)$

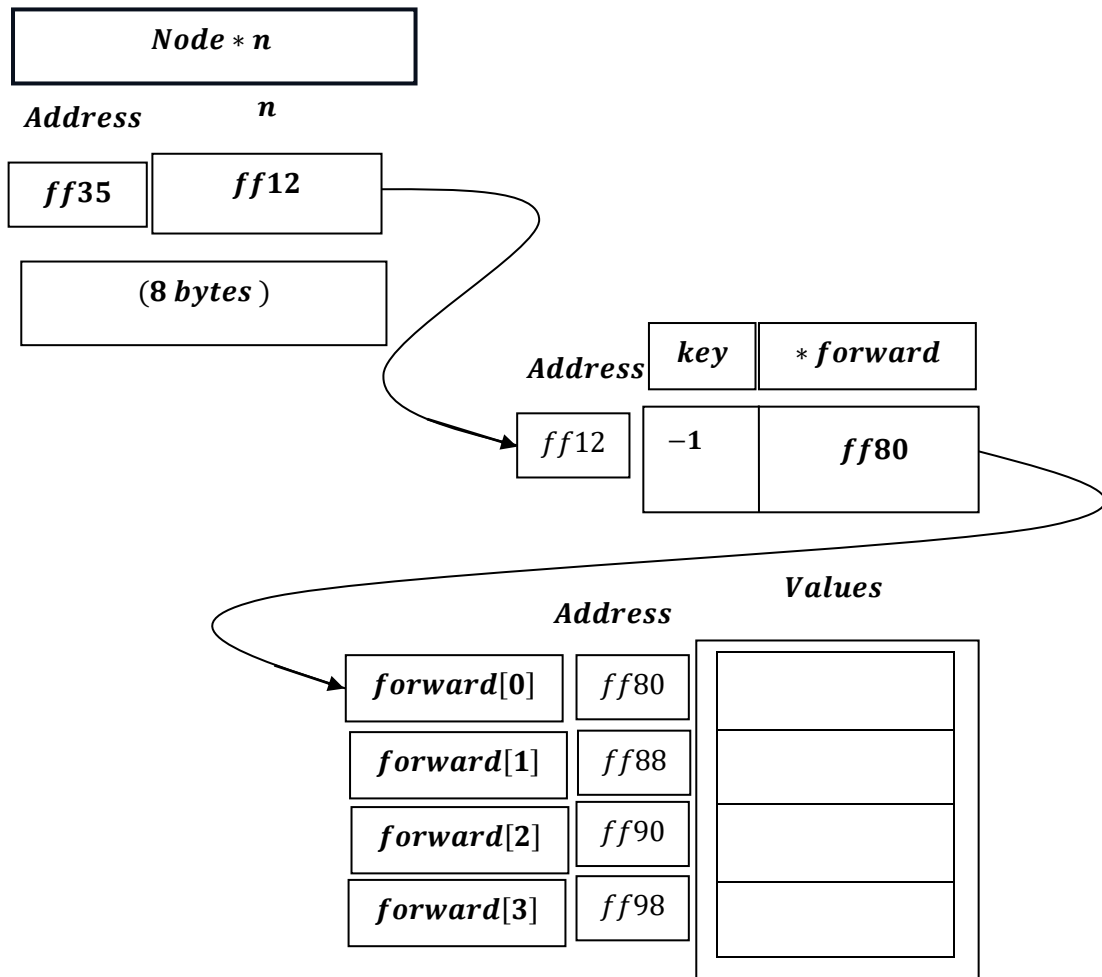
$= 8 \text{ bytes} \times 4$

$= 32 \text{ bytes}$

Which means , it allocates 4 slots of memory of 8 bytes , like: –



And now,



Actual structure

Address at which value is stored	Value	offset used
ff20	key = -1	key[ff20 – ff23] 4 bytes for int
		padding → [ff24 – ff27] 4 bytes for padding
ff28	Address: ff80	* forward → [ff28 – ff2f] 8 bytes for pointer variable
ff80	NULL	forward[0] → [ff80 – ff87] 8 bytes for pointer variable
ff88	NULL	forward[1] → [ff88 – ff8f] 8 bytes for pointer variable
ff90	NULL	forward[2] → [ff90 – ff97] 8 bytes for pointer variable
ff98	NULL	forward[3] → [ff98 – ff9f] 8 bytes for pointer variable

This will be discussed in memory allocation section.

```
for (int i = 0; i <= level; i++) {  
    n->forward[i] = NULL;  
}
```

Note , level = MAXL_EVEL = 3 .

Iteration 1:

$i = 0; 0 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[0] = \text{NULL}$

$i = 0 + 1 = 1$

Iteration 2:

$i = 1; 1 \leq \text{level} = 3$ [Condition True]

$n \rightarrow \text{forward}[1] = \text{NULL}$

$i = 1 + 1 = 2$

Iteration 3:

$i = 2; 2 \leq level = 3$ [Condition True]

$n \rightarrow forward[2] = NULL$

$i = 2 + 1 = 3$

Iteration 4:

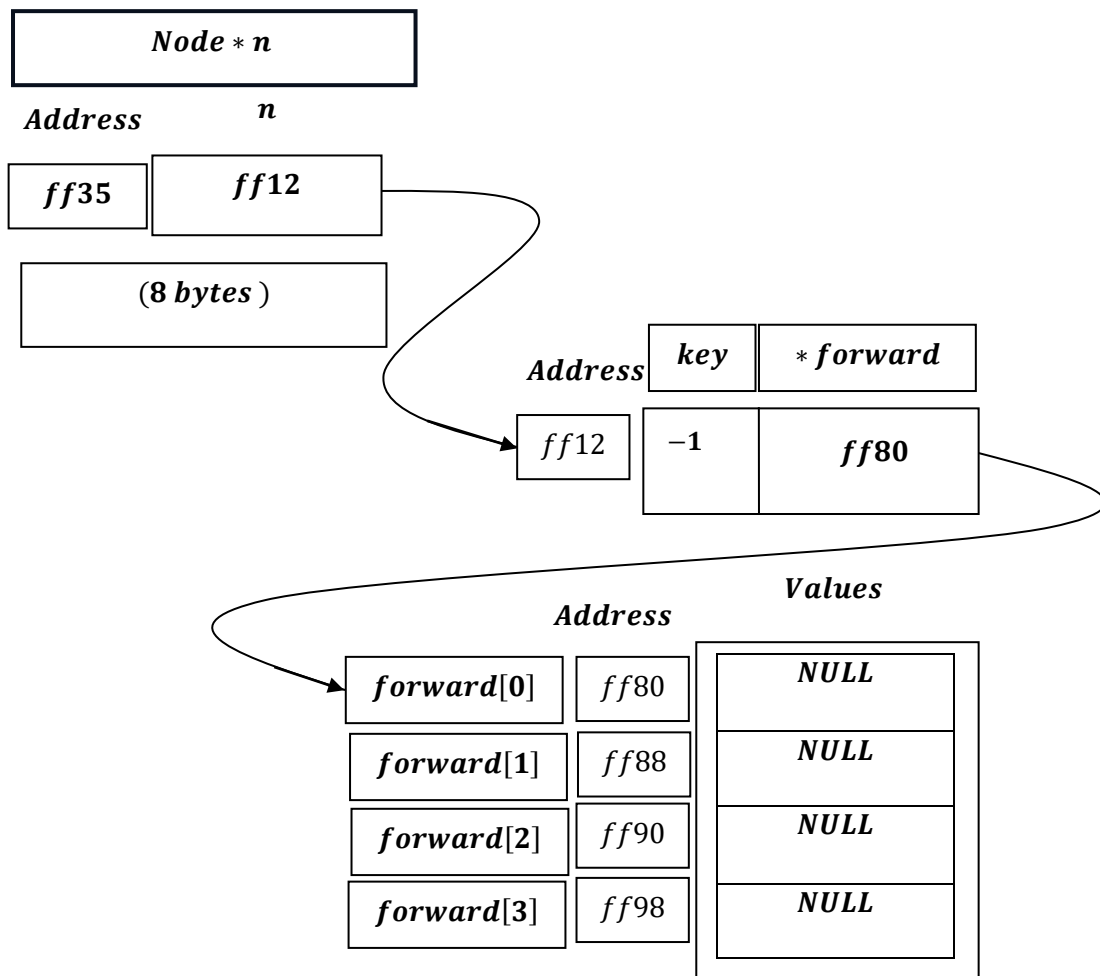
$i = 3; 3 \leq level = 3$ [Condition True]

$n \rightarrow forward[3] = NULL$

$i = 3 + 1 = 4$

Iteration 5:

$i = 4; 4 \leq level = 3$ [Condition is False]
hence, loop exits

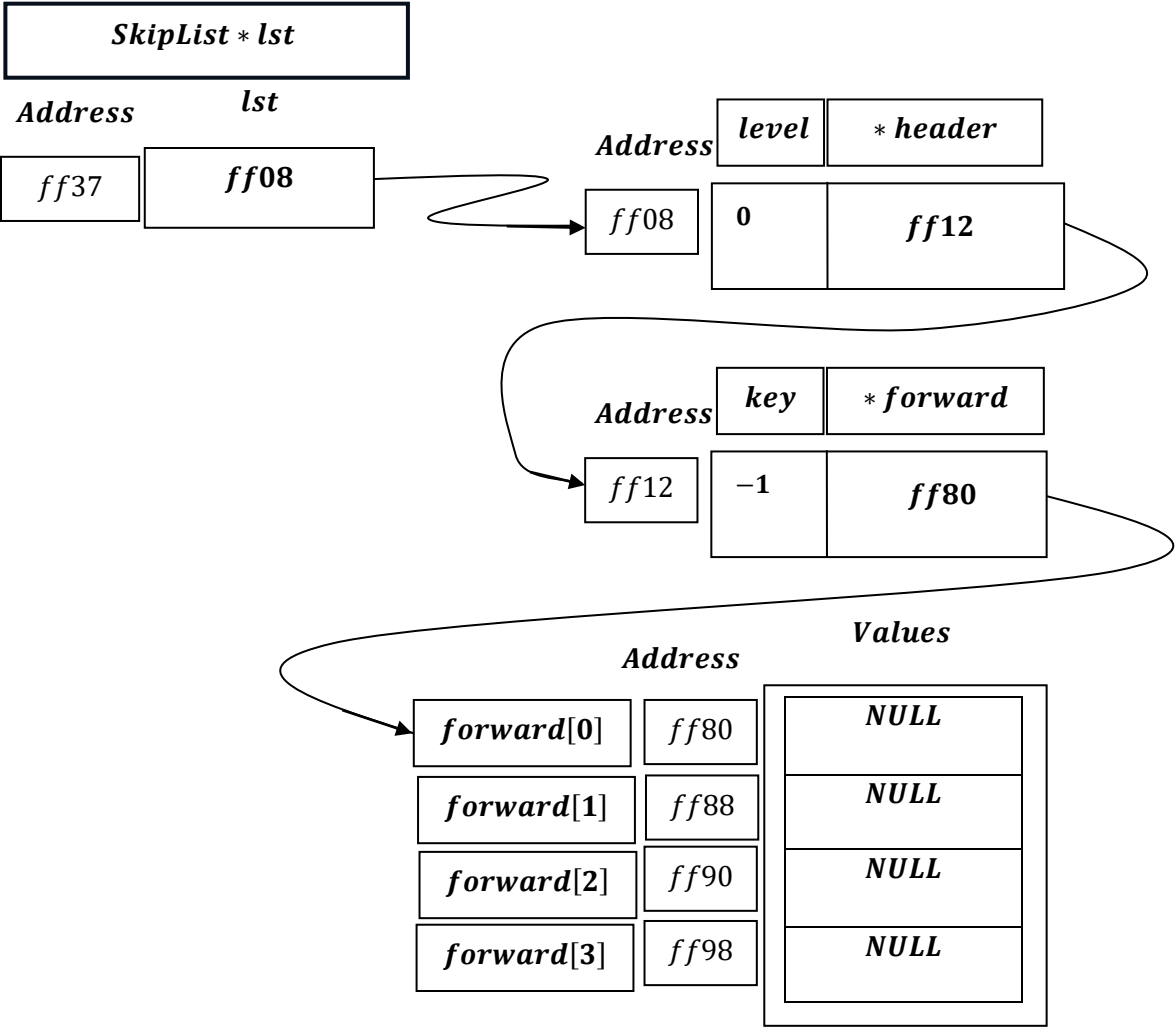


```
return n;
```

It returns address value that pointer variable n has stored : ff12

```
lst->header = createNode(-1, MAX_LEVEL);
```

i.e. lst → header = ff12.

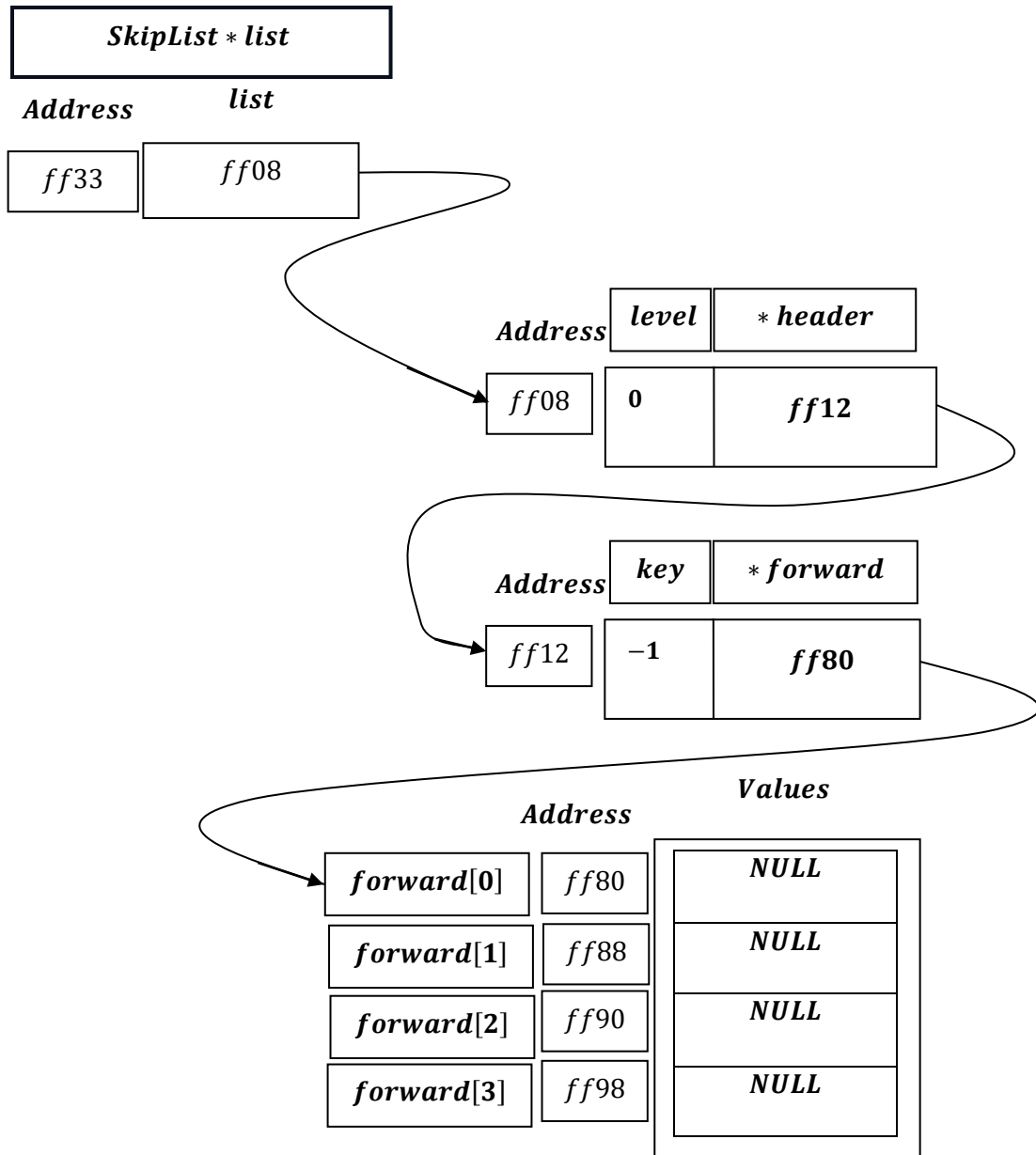


```
return lst;
```

return lst = ff08.

```
struct SkipList *list = createSkipList();
```

*struct SkipList * list = ff08.*



Time Complexity

```
#define MAX_LEVEL 3
```

#define is a preprocessor directive, not an executable statement.

Before compilation, the C preprocessor simply replaces every occurrence of MAX_LEVEL with 3.

There is no runtime execution, so there is no runtime time complexity, and the compiler simply records it.

Preprocessor Directive, Not Runtime Code: #define is handled by the C/C++ preprocessor before the compiler even sees the code. It acts essentially like a find and replace operation. Every time we use MAX_LEVEL in your code, the preprocessor swaps it out with the literal number 3.

Time Complexity: $O(1)$ (or zero runtime cost). Because the substitution happens at compile time, there is no code generated to execute or evaluate this definition while the program is running.

The runtime time complexity of the definition itself is practically zero.

Space Complexity: $O(1)$. Unlike declaring a variable (e.g., `const int MAX_LEVEL = 3;` which might allocate memory depending on compiler optimizations), #define does not allocate any memory in the RAM for the value 3. The literal number 3 is just baked directly into the machine code instructions wherever it is used. Therefore, it requires no extra runtime memory allocation, making the space complexity $O(1)$.

```
// Node structure
struct Node {
    int key;
    // Array of pointers to next nodes, one for each level
    struct Node **forward;
};
```

*Declaring int level; and struct Node ** forward; takes constant time complexity : $O(1)$, no runtime work is performed, the compiler simply records the layout of the structure.*

Space Complexity

The structure contains:

- *int key* → (4 bytes for both 64/32 bit architecture) $O(1)$
- *struct Node ** forward (a pointer)* → (8 bytes for 64 bit architecture and 4 bytes for 32 bit architecture) $O(1)$

Therefore, the space complexity of Node objects are: $O(1)$.

```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

*Declaring int level; and struct Node * header; takes constant time complexity : $O(1)$, no runtime work is performed, the compiler simply records the layout of the structure.*

Space Complexity

The structure contains:

- *int level* → (4 bytes for both 64/32 bit architecture) $O(1)$
- *struct Node * header (a pointer)* → (8 bytes for 64 bit architecture and 4 bytes for 32 bit architecture) $O(1)$

Therefore, the space complexity of SkipList objects are: $O(1)$.

```

// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

```

Taking into account the constant #define MAX_LEVEL 3.

```

struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));

```

Allocation of memory for SkipList struct takes constant time. $\rightarrow O(1)$

```

lst->level = 0;

```

This assignment ``lst  $\rightarrow$  level = 0`` takes constant amount of time. $\rightarrow O(1)$

```

lst->header = createNode(-1, MAX_LEVEL);

```

createNode(-1, MAX_LEVEL)

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

$key = -1$ and $level = MAX_LEVEL = 3$ $\left[\begin{matrix} \text{constant time} \\ \text{complexity} \end{matrix} \right] \rightarrow \text{Takes } O(c) \text{ or } O(1)$

```
struct Node *n = (struct Node *)malloc(sizeof(struct Node));
```

Allocation of memory for SkipList struct takes constant time. $\rightarrow$ Takes $O(c)$ or $O(1)$

```
n->key = key;
```

$n \rightarrow key = key = -1$

This assignment takes constant amount of time. $\rightarrow O(c)$ or $O(1)$

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));
```

Allocation of memory for SkipList struct takes constant time. $\rightarrow$ Takes $O(c)$ or $O(1)$

```
// Initialize pointers to NULL
for (int i = 0; i <= level; i++) {
    n->forward[i] = NULL;
}
```

Here, we can say $\rightarrow O(LEVEL)$, but we know, $level = MAX_LEVEL = 3$ $\left[\begin{matrix} LEVEL \\ is \\ Constant \end{matrix} \right]$

As, i runs from 0 to $level = MAX_LEVEL = 3$ (which is a constant)

Loop becomes constant i.e. $O(c)$ or $O(1)$.

Retrun `n` which also takes constant `O(c)` or $O(1)$.

Hence in both the cases :

```
// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Time Complexity $\rightarrow O(1)$.

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

Time Complexity $\rightarrow O(LEVEL) \approx O(MAX_LEVEL) \approx O(c) \approx O(1)$.

Space Complexity of createNode

```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    // elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
    + 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}
```

- **Parametric Space (Input Space): $O(1)$**

The function takes two integer parameters (key and level) by value, which require a fixed amount of stack memory regardless of what those values are.

- **Full Data Input Space: $O(1)$**

Because the function accepts only primitive types and no externally allocated arrays or structures, the full input space is identical to the parametric space.

- **Auxiliary Space: $O(1)$**

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

The function allocates heap memory dynamically. While the Node wrapper requires constant $O(1)$ space, the forward array requires memory proportional to $level + 1$.

As it require `level + 1`, we can say space complexity = $O(level)$, but :

As, $level = MAX_LEVEL = 3$ is a constant ,

The function allocates dynamic memory on the heap.

Because $level \leq 3$, the forward array will allocate a maximum of 4 pointers.

Allocating a fixed maximum amount of memory , regardless of the overall size of the skip list, is an $O(1)$ operation.

```
// Initialize pointers to NULL
for (int i = 0; i <= level; i++) {
    n->forward[i] = NULL;
}
```

here int `i` and `level` as well as $n \rightarrow forward[i] = NULL$ using same constant amount of space. Hence no new space is created like recursion , therefore this loop also consumes constant space = $O(c)$ or $O(1)$.

Hence Auxiliary space also becomes $O(1)$.

- **Total Space Complexity = Input Space + Auxiliary Space = $O(1)$.**

Space Complexity of createSkipList

```
// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}
```

Because MAX_LEVEL is defined as a fixed constant (3), the mathematical complexity of this specific function simplifies to constant time and space.

Space Complexity Breakdown:

- *Parametric Space (Input Space): $O(1)$*

The function takes no arguments, meaning parameter space is strictly zero/constant.

- *Full Data Input Space: $O(1)$*

No inputs are passed, so this remains constant.

- *Auxiliary Space: $O(1)$*

The function allocates a fixed – size SkipList struct on the heap.

The call to createNode(-1, MAX_LEVEL) allocates memory for a Node and an array of 4 pointers ($\text{MAX_LEVEL} + 1$).

Because this allocation size is fixed at compile – time and does not grow, the auxiliary space is strictly $O(1)$.

Total Space Complexity: Input Space + Auxiliary Space = $O(1)$.

Algorithm

CONSTANT MAX_LEVEL = 3

1. Function createSkipList()

**((SkipList *)malloc(sizeof(SkipList)) is availability of space for insertion of Data,
hence, lets name it AVAIL.**

2. SET LST_PTR := AVAIL

3. SET LEVEL[LST_PTR] := 0

4. SET HEADER[LST_PTR] = createNode(-1, MAX_LEVEL)

Translatin createNode(-1, MAX_LEVEL)

1. Function createNode(Set Key := -1, Set Level := MAX_LEVEL)

**(Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, lets name it AVAIL.**

2. SET n_ptr := AVAIL.

3. SET key[n_ptr] := Key

4. SET forward[n_ptr] := AVAIL * (Level + 1)

5. Repeat for $i = 0$ and $i \leq \text{Level}$:

5.a. Set forward[n_ptr][i] := NULL

[End of For Loop]

6. Return n_ptr

[End of Function createNode]

5. Return LST_PTR

[End of Function createSkipList]

[End of Algorithm]

Note :

We can also write in algorithm:

1. **SET $n_ptr.key := Key$**
 2. **SET $n_ptr.forward := AVAIL * (Level + 1)$**
 3. **Repeat for $i = 0$ and $i \leq Level$:**
 5. a. **Set $n_ptr.forward[i] := NULL$**
- [End of For Loop]**

Also fine .
